Data Mining Project

Sleep Disorder Analysis

Preethi Sri B - 23MIS0470

Abstract:

This project aims to analyze and classify datasets using various supervised learning methods to study model performance and statistical behavior. A part of the reduced dataset-subset_reduced_dataset.xlsx with the dimensionalities of attributes A to F-was used for preprocessing, scaling of features, and classification. It uses major classifiers-Decision Tree, SVM, Naive Bayes, and KNN-to effectively predict target classes. Each model performance was evaluated by confusion matrices, statistical measures, and ROC curve analysis. This paper aims to consider the most accurate and reliable classifier while visualizing the efficiency in combined metrics and ROC plots for the chosen models. This work demonstrates how different classifiers behave on the same dataset and highlights the importance of preprocessing and model evaluation for successful predictive modeling in data mining.

Preprocessing:

In order to classify the dataset, a series of preprocessing steps were required. The data was imported from an Excel file and studied to be complete and consistent. Label encoding was applied to any categorical attributes so that they were numerical for the algorithms in machine learning. Features A-F columns selected were standardized by StandardScaler; hence, the scale for all attributes was such that each contributed equally to the model with zero mean and unit variance. The dataset was then split into training and test subsets in a 70:30 ratio, hence allowing reliable model evaluations. This preprocessing pipeline thereby cleaned and scaled the data for effective classification and performance comparison.

Classifiers :

Four different supervised learning algorithms were implemented: Decision Tree, SVM, Naive Bayes, and KNN. In the Decision Tree classifier, the recursive splitting of data with respect to information gain gives a hierarchical structure for classification. The SVM classifier separates data by an optimal hyperplane such that the margin between classes is highest. The Naive Bayes classifier applies Bayes' theorem with an assumption of feature independence to offer a computationally efficient model for probabilistic classification. KNN predicts the target class of an example based on the majority vote of its k closest neighbors in the feature space. All the classifiers were trained with the same set of training data and tested with the same test data to measure the fair performance comparison among them.

Evaluation:

Performance metrics for each classifier included Accuracy, Precision, Recall (Sensitivity), Specificity, F-Measure, and Error Rate. Additionally, True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) were calculated for every model. Confusion matrices were generated for each classifier to plot a visual representation of the performance of classification, including a summary confusion matrix afterwards that combined all the models' results. An ROC curve showing the models' discriminative power was plotted, which highlighted visually the trade-off between sensitivity and specificity. In this way, full comprehension of the strengths and weaknesses of each classifier was obtained, enabling the determination of which model best fit the characteristics of the dataset.

Template:

Phase 1 –

- Pre-Processing
  - Data Cleaning
  - Data Integration
  - Data Transformation
  - Date Reduction

- Statistical Measures

Phase 2 –

- Classifiers
  - KNN
  - Naïve Bayes
  - Decision Tree

Phase 3 –

- Accuracy
- Precision
- Error Rate
- Sensitivity
- Specificity
- F-Measure

Pre-Processing:

--- Data Cleaning ---

Code:

```
import pandas as pd
file_path = r"C:\Users\User\OneDrive\Documents\DataMining Proj\sleep_disorder_dataset.xlsx"
df = pd.read_excel(file_path, sheet_name="Sheet1")
df.drop_duplicates(inplace=True)
df.dropna(subset=["Patient_ID"], inplace=True)
df["Age"].fillna(df["Age"].median(), inplace=True)
df["AHI_Score"].fillna(df["AHI_Score"].median(), inplace=True)
df["SaO2_Level"].fillna(df["SaO2_Level"].median(), inplace=True)
df["Gender"].fillna(df["Gender"].mode()[0], inplace=True)
df["Sleep_Disorder_Type"].fillna(df["Sleep_Disorder_Type"].mode()[0], inplace=True)
df["Gender"] = df["Gender"].str.strip().str.title()  # Male/Female
df["Sleep_Disorder_Type"] = df["Sleep_Disorder_Type"].str.strip().str.title()
df["Diagnosis_Confirmed"] = df["Diagnosis_Confirmed"].astype(int)
if "OCR_Extracted_Text" in df.columns:
    df.drop(columns=["OCR_Extracted_Text"], inplace=True)
cleaned_file = r"C:\Users\User\OneDrive\Documents\DM\cleaned_sleep_disorder_dataset.xlsx"
df.to_excel(cleaned_file, index=False)
print("Data cleaning complete! Cleaned file saved as:", cleaned_file)
```

--- Data Integration ---

Code:

```
import pandas as pd
sleep_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\cleaned_sleep_disorder_dataset.xlsx"
lifestyle_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\patient_lifestyle_data.xlsx"
sleep_df = pd.read_excel(sleep_file)
lifestyle_df = pd.read_excel(lifestyle_file)
sleep_df["Patient_ID"] = sleep_df["Patient_ID"].astype(str).str.strip()
lifestyle_df["Patient_ID"] = lifestyle_df["Patient_ID"].astype(str).str.strip()
print("Unique IDs in sleep dataset:", len(sleep_df["Patient_ID"].unique()))
print("Unique IDs in lifestyle dataset:", len(lifestyle_df["Patient_ID"].unique()))
common_ids = set(sleep_df["Patient_ID"]).intersection(set(lifestyle_df["Patient_ID"]))
print("Number of common Patient_IDs:", len(common_ids))
integrated_df = pd.merge(sleep_df, lifestyle_df, on="Patient_ID", how="inner")
integrated_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\integrated_dataset.xlsx"
integrated_df.to_excel(integrated_file, index=False)
print("Integration complete! Rows after merge:", len(integrated_df);
```

--- Data Transformation ---

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
integrated_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\integrated_dataset.xlsx"
df = pd.read_excel(integrated_file)
scaler = MinMaxScaler()
numeric_cols = ["Age", "AHI_Score", "SaO2_Level"]
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
label_enc = LabelEncoder()
categorical_cols = ["Gender", "Sleep_Disorder_Type", "Smoking_Habit",
"Alcohol_Consumption", "Caffeine_Intake", "Work_Shift"]
for col in categorical_cols:
    df[col] = label_enc.fit_transform(df[col].astype(str))
df["Age_Group"] = pd.cut(
    df["Age"],
    bins=[0, 0.3, 0.6, 1.0],
    labels=["Young", "Middle", "Old"]
)
transformed_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\transformed_dataset.xlsx"
df.to_excel(transformed_file, index=False)
print("Data transformation complete! Transformed file saved as:", transformed_file)
```

--- Data Reduction ---

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
integrated_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\integrated_dataset.xlsx"
df = pd.read_excel(integrated_file)
scaler = MinMaxScaler()
numeric_cols = ["Age", "AHI_Score", "SaO2_Level"]
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
label_enc = LabelEncoder()
categorical_cols = ["Gender", "Sleep_Disorder_Type", "Smoking_Habit",
"Alcohol_Consumption", "Caffeine_Intake", "Work_Shift"]
for col in categorical_cols:
    df[col] = label_enc.fit_transform(df[col].astype(str))
df["Age_Group"] = pd.cut(
    df["Age"],
    bins=[0, 0.3, 0.6, 1.0],
    labels=["Young", "Middle", "Old"]
)
```

```
transformed_file = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\transformed_dataset.xlsx"
df.to_excel(transformed_file, index=False)
print("Data transformation complete! Transformed file saved as:", transformed_file)
```

Statistical Measures:

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
import scipy.stats as stats
from scipy.spatial import distance
from scipy.spatial.distance import pdist, squareform
import gower
from sklearn.preprocessing import LabelEncoder
file_path = r"C:\Users\User\OneDrive\Documents\DataMining Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)
df_AF = df.iloc[:, 0:6]
df_selected = df.iloc[:, 0:6]
print("----- Central Tendency Measures -----")
print("\nMean values:")
print(df.mean(numeric_only=True))
print("\nMedian values:")
print(df.median(numeric_only=True))
print("\nMode values:")
print(df.mode(numeric_only=True).iloc[0])
print("----- Range, Quartiles & IQR -----")
print("\nRange (Max - Min):")
range_values = df.max(numeric_only=True) - df.min(numeric_only=True)
print(range_values)
print("\nQuartiles (Q1, Q2, Q3):")
Q1 = df.quantile(0.25, numeric_only=True)
Q2 = df.quantile(0.50, numeric_only=True)
Q3 = df.quantile(0.75, numeric_only=True)
print("Q1:\n", Q1)
print("Q2 (Median):\n", Q2)
print("Q3:\n", Q3)
print("\nInterquartile Range (IQR):")
IQR = Q3 - Q1
print(IQR)
print("----- Five Number Summary -----")
five_num_summary = pd.DataFrame({
    "Minimum": df.min(numeric_only=True),
    "Q1 (25%)": df.quantile(0.25, numeric_only=True),
```

```
    "Median (Q2)": df.median(numeric_only=True),
    "Q3 (75%)": df.quantile(0.75, numeric_only=True),
    "Maximum": df.max(numeric_only=True)
})
print(five_num_summary)
numeric_cols = df.select_dtypes(include=['number']).columns
print("----- Outlier Detection using IQR -----")
outliers_dict = {}
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
    outliers_dict[col] = outliers.values
    print(f"\n{col}:")
    print(f"Lower Bound = {lower_bound}, Upper Bound = {upper_bound}")
    print(f"Outliers: {outliers.values if len(outliers) > 0 else 'None'}")
plt.figure(figsize=(12, 6))
df[numeric_cols].boxplot()
plt.title("Boxplots with Outliers (shown as dots)")
plt.xticks(rotation=45)
plt.show()
print("----- Variance -----")
print(df_AF.var(numeric_only=True))
print("\n----- Standard Deviation -----")
print(df_AF.std(numeric_only=True))
print("----- Covariance Matrix -----")
print(df_AF.cov())
print("\n----- Correlation Matrix -----")
print(df_AF.corr())
contingency_table = pd.crosstab(df.iloc[:,0], df.iloc[:,1])
chi2, p, dof, expected = chi2_contingency(contingency_table)
print("Chi-Square Test Results:")
print("Chi2 Value:", chi2)
print("p-value:", p)
print("Degrees of Freedom:", dof)
for column in df_selected.columns:
    data = df_selected[column].dropna()
    quantiles = np.linspace(0, 1, len(data))
    sorted_data = np.sort(data)
    plt.figure(figsize=(8, 5))
    plt.plot(quantiles, sorted_data, 'o', markersize=3)
    plt.title(f"Quantile Plot for {column}")
    plt.xlabel("Quantiles (0 to 1)")
    plt.ylabel(column)
    plt.grid(True)
```

```
    plt.show()
for col in df_AF.columns:
    stats.probplot(df_AF[col], dist="norm", plot=plt)
    plt.title(f"Q-Q Plot for {col}")
    plt.show()
df_AF.hist(figsize=(12, 8), bins=15)
plt.suptitle("Histograms of A-F Columns")
plt.show()
pd.plotting.scatter_matrix(df_AF, figsize=(12, 8), diagonal='hist')
plt.suptitle("Scatter Matrix Plot")
plt.show()
print("----- Pearson Correlation -----")
print(df_AF.corr(method='pearson'))
print("\n----- Cosine Similarity -----")
cos_sim = cosine_similarity(df_AF.T)
cosine_df = pd.DataFrame(cos_sim, index=df_AF.columns, columns=df_AF.columns)
print(cosine_df)
row1 = df_AF.iloc[0]
row2 = df_AF.iloc[1]
minkowski_dist = distance.minkowski(row1, row2, p=3)
print("Minkowski Distance (p=3) between Row1 & Row2:", minkowski_dist)
euclidean = distance.euclidean(row1, row2)
manhattan = distance.cityblock(row1, row2)
supremum = distance.chebyshev(row1, row2)
print("Euclidean Distance:", euclidean)
print("Manhattan Distance:", manhattan)
print("Supremum (Chebyshev) Distance:", supremum)
euclidean_matrix = squareform(pdist(df_AF, metric='euclidean'))
print("Euclidean Dissimilarity Matrix:\n", euclidean_matrix)
manhattan_matrix = squareform(pdist(df_AF, metric='cityblock'))
print("\nManhattan Dissimilarity Matrix:\n", manhattan_matrix)
supremum_matrix = squareform(pdist(df_AF, metric='chebyshev'))
print("\nSupremum (Chebyshev) Dissimilarity Matrix:\n", supremum_matrix)
df_nominal = df_selected.astype(str).apply(LabelEncoder().fit_transform)
def simple_matching(u, v):
    return np.sum(u != v) / len(u)
nominal_dist = squareform(pdist(df_nominal.values, metric=simple_matching))
print("\nNominal/Categorical (Simple Matching):")
print(pd.DataFrame(nominal_dist).round(2))
df_binary = df_selected.applymap(lambda v: 1 if v > df_selected.median().median() else 0)
binary_dist = squareform(pdist(df_binary.values, metric="jaccard"))
print("\nBinary (Jaccard Distance):")
print(pd.DataFrame(binary_dist).round(2))
df_ordinal = df_selected.rank(method="average") / df_selected.rank(method="average").max()
ordinal_dist = squareform(pdist(df_ordinal.values, metric="euclidean"))
print("\nOrdinal (Euclidean on Ranks):")
print(pd.DataFrame(ordinal_dist).round(2))
gower_dist = gower.gower_matrix(df_selected)
```

```
gower_dist = np.round(gower_dist, 2)
print("\n---Mixed Attributes---")
print(pd.DataFrame(gower_dist))
```

Output:

----- Central Tendency Measures -----

```
Mean values:
Age                    0.486883
Gender                 0.508000
Sleep_Disorder_Type    2.055000
AHI_Score              0.514212
SaO2_Level             0.500516
Diagnosis_Confirmed    0.884000
Smoking_Habit          0.474000
Alcohol_Consumption    1.012000
Exercise_Frequency     3.438000
Caffeine_Intake        1.010000
Work_Shift             0.971000
dtype: float64
```

```
Median values:
Age                    0.480519
Gender                 1.000000
Sleep_Disorder_Type    3.000000
AHI_Score              0.527817
SaO2_Level             0.505613
Diagnosis_Confirmed    1.000000
Smoking_Habit          0.000000
Alcohol_Consumption    1.000000
Exercise_Frequency     3.000000
Caffeine_Intake        1.000000
Work_Shift             1.000000
dtype: float64
```

```
Mode values:
Age                    0.792208
Gender                 1.000000
Sleep_Disorder_Type    3.000000
AHI_Score              0.129678
SaO2_Level             0.524860
Diagnosis_Confirmed    1.000000
Smoking_Habit          0.000000
Alcohol_Consumption    2.000000
Exercise_Frequency     4.000000
Caffeine_Intake        2.000000
Work_Shift             1.000000
Name: 0, dtype: float64
```

----- Range, Quartiles & IQR -----

```
Range (Max - Min):
Age                    1.0
Gender                 1.0
Sleep_Disorder_Type    4.0
AHI_Score              1.0
SaO2_Level             1.0
Diagnosis_Confirmed    1.0
Smoking_Habit          1.0
Alcohol_Consumption    2.0
Exercise_Frequency     7.0
Caffeine_Intake        2.0
Work_Shift             2.0
dtype: float64
```

```
Quartiles (Q1, Q2, Q3):
Q1:
 Age                    0.233766
Gender                  0.000000
Sleep_Disorder_Type     0.000000
AHI_Score               0.268761
SaO2_Level              0.254010
Diagnosis_Confirmed     1.000000
Smoking_Habit           0.000000
Alcohol_Consumption     0.000000
Exercise_Frequency      1.000000
Caffeine_Intake         0.000000
Work_Shift              0.000000
Name: 0.25, dtype: float64
Q2 (Median):
 Age                    0.480519
Gender                  1.000000
Sleep_Disorder_Type     3.000000
AHI_Score               0.527817
SaO2_Level              0.505613
Diagnosis_Confirmed     1.000000
Smoking_Habit           0.000000
Alcohol_Consumption     1.000000
Exercise_Frequency      3.000000
Caffeine_Intake         1.000000
Work_Shift              1.000000
Name: 0.5, dtype: float64
```

```
Q3:
 Age                    0.740260
Gender                  1.000000
Sleep_Disorder_Type     3.000000
AHI_Score               0.762157
SaO2_Level              0.753408
Diagnosis_Confirmed     1.000000
Smoking_Habit           1.000000
Alcohol_Consumption     2.000000
Exercise_Frequency      5.000000
Caffeine_Intake         2.000000
Work_Shift              2.000000
Name: 0.75, dtype: float64

Interquartile Range (IQR):
Age                     0.506494
Gender                  1.000000
Sleep_Disorder_Type     3.000000
AHI_Score               0.493396
SaO2_Level              0.499399
Diagnosis_Confirmed     0.000000
Smoking_Habit           1.000000
Alcohol_Consumption     2.000000
Exercise_Frequency      4.000000
Caffeine_Intake         2.000000
Work_Shift              2.000000
dtype: float64
```

----- Five Number Summary -----

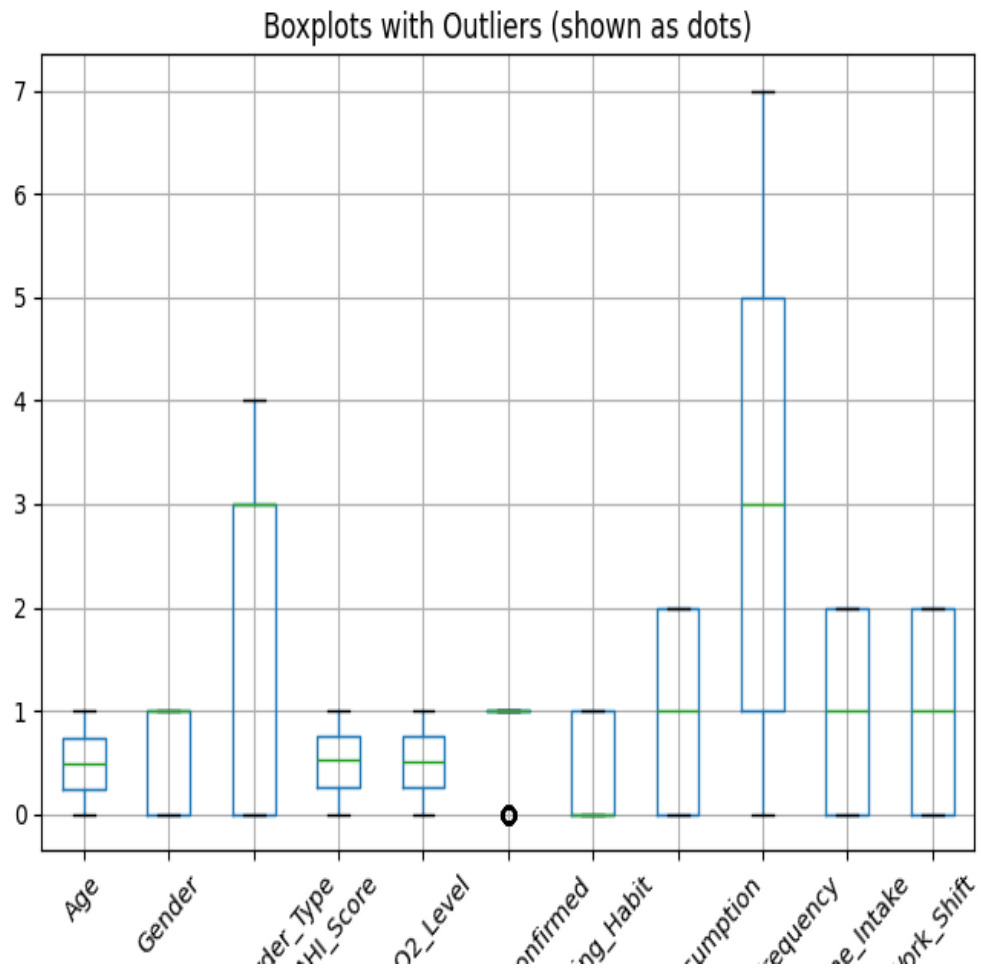
```
----- Five Number Summary -----
                     Minimum  Q1 (25%)  Median (Q2)  Q3 (75%)  Maximum
Age                      0.0  0.233766     0.480519  0.740260      1.0
Gender                   0.0  0.000000     1.000000  1.000000      1.0
Sleep_Disorder_Type      0.0  0.000000     3.000000  3.000000      4.0
AHI_Score                0.0  0.268761     0.527817  0.762157      1.0
SaO2_Level               0.0  0.254010     0.505613  0.753408      1.0
Diagnosis_Confirmed      0.0  1.000000     1.000000  1.000000      1.0
Smoking_Habit            0.0  0.000000     0.000000  1.000000      1.0
Alcohol_Consumption      0.0  0.000000     1.000000  2.000000      2.0
Exercise_Frequency       0.0  1.000000     3.000000  5.000000      7.0
Caffeine_Intake          0.0  0.000000     1.000000  2.000000      2.0
Work_Shift               0.0  0.000000     1.000000  2.000000      2.0
```

----- Outlier Detection using IQR -----

```
----- Outlier Detection using IQR -----

Age:
Lower Bound = -0.525974025974026, Upper Bound = 1.5
Outliers: None

Gender:
Lower Bound = -1.5, Upper Bound = 2.5
Outliers: None

Sleep_Disorder_Type:
Lower Bound = -4.5, Upper Bound = 7.5
Outliers: None

AHI_Score:
Lower Bound = -0.4713327996798081, Upper Bound = 1.5022513508104867
Outliers: None

SaO2_Level:
Lower Bound = -0.4950882117080991, Upper Bound = 1.5025060144346436
Outliers: None
```

```
Diagnosis_Confirmed:
Lower Bound = 1.0, Upper Bound = 1.0
Outliers: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0]

Smoking_Habit:
Lower Bound = -1.5, Upper Bound = 2.5
Outliers: None

Alcohol_Consumption:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None

Exercise_Frequency:
Lower Bound = -5.0, Upper Bound = 11.0
Outliers: None

Caffeine_Intake:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None

Work_Shift:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None
```

----BoxPlot----



----- Variance -----

```
----- Variance -----
Age                    0.086601
Gender                 0.250186
Sleep_Disorder_Type    2.290265
AHI_Score              0.082180
SaO2_Level             0.082866
Diagnosis_Confirmed    0.102647
dtype: float64
```

----- Standard Deviation -----

```
----- Standard Deviation -----
Age                    0.294281
Gender                 0.500186
Sleep_Disorder_Type    1.513362
AHI_Score              0.286670
SaO2_Level             0.287865
Diagnosis_Confirmed    0.320385
dtype: float64
```

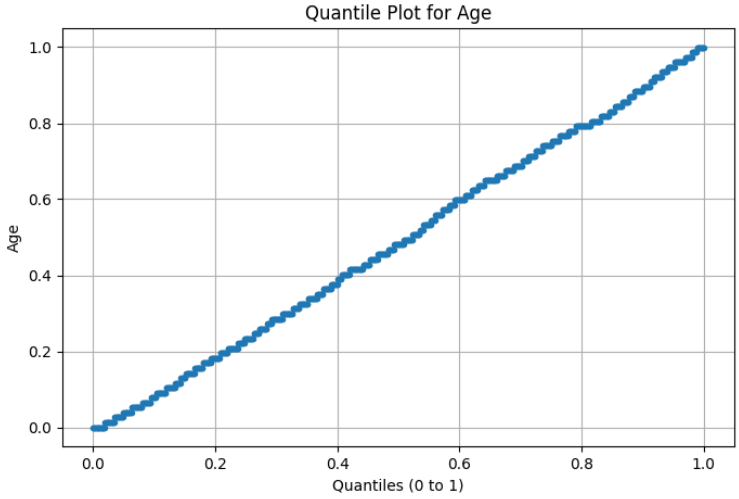
----- Covariance Matrix -----

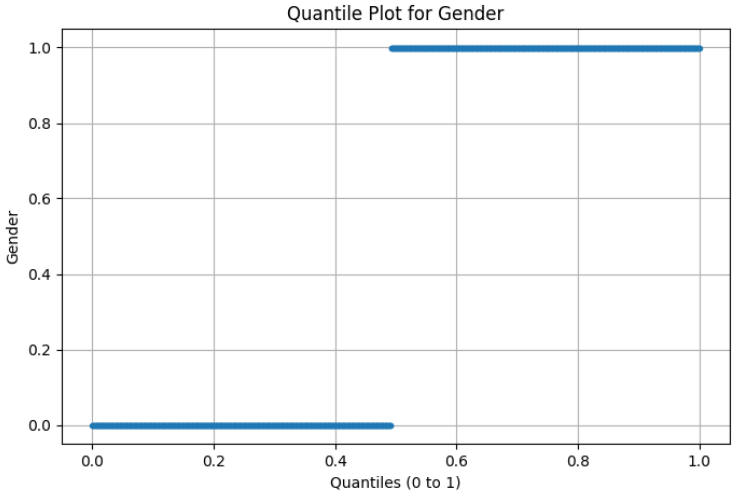
```
----- Covariance Matrix -----
                        Age    Gender  Sleep_Disorder_Type  AHI_Score  SaO2_Level  Diagnosis_Confirmed
Age                0.086601  0.002094            -0.006343   0.000773    0.001456            -0.000574
Gender             0.002094  0.250186            -0.007948  -0.000813    0.005833            -0.011083
Sleep_Disorder_Type -0.006343 -0.007948             2.290265  -0.005392   -0.029732             0.008388
AHI_Score          0.000773 -0.000813            -0.005392   0.082180   -0.002168             0.043055
SaO2_Level         0.001456  0.005833            -0.029732  -0.002168    0.082866            -0.032164
Diagnosis_Confirmed -0.000574 -0.011083             0.008388   0.043055   -0.032164             0.102647
```

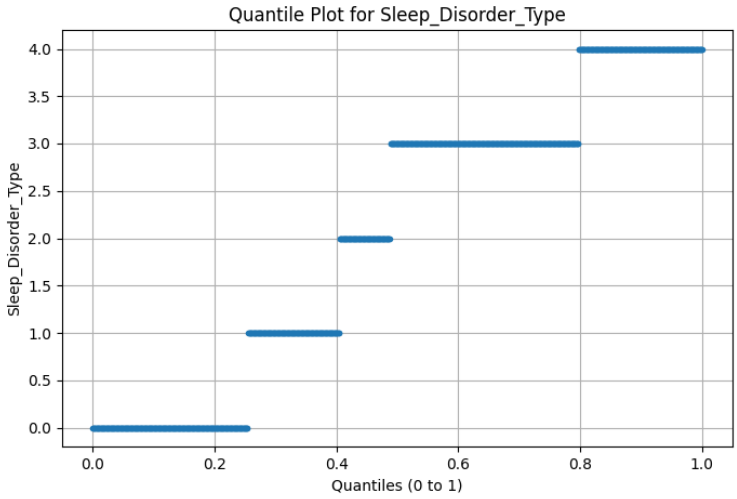
----- Correlation Matrix -----

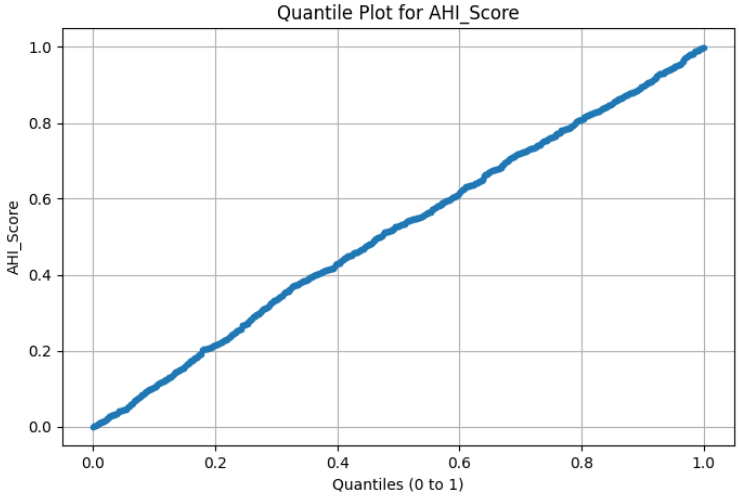
```
----- Correlation Matrix -----
                        Age    Gender  Sleep_Disorder_Type  AHI_Score  SaO2_Level  Diagnosis_Confirmed
Age                1.000000  0.014226            -0.014243   0.009158    0.017190            -0.006089
Gender             0.014226  1.000000            -0.010500  -0.005671    0.040510            -0.069160
Sleep_Disorder_Type -0.014243 -0.010500             1.000000  -0.012429   -0.068248             0.017301
AHI_Score          0.009158 -0.005671            -0.012429   1.000000   -0.026270             0.468780
SaO2_Level         0.017190  0.040510            -0.068248  -0.026270    1.000000            -0.348742
Diagnosis_Confirmed -0.006089 -0.069160             0.017301   0.468780   -0.348742             1.000000
Chi-Square Test Results:
Chi2 Value: 73.7894421072609
p-value: 0.5825637929794798
Degrees of Freedom: 77
```

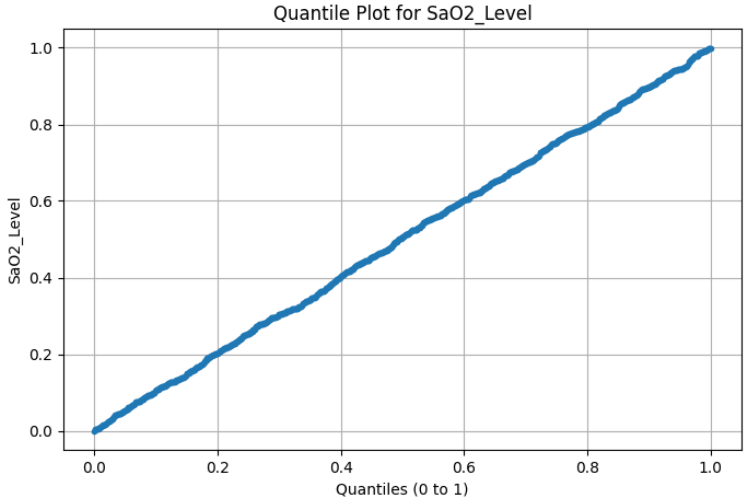
---Quantile Plot---

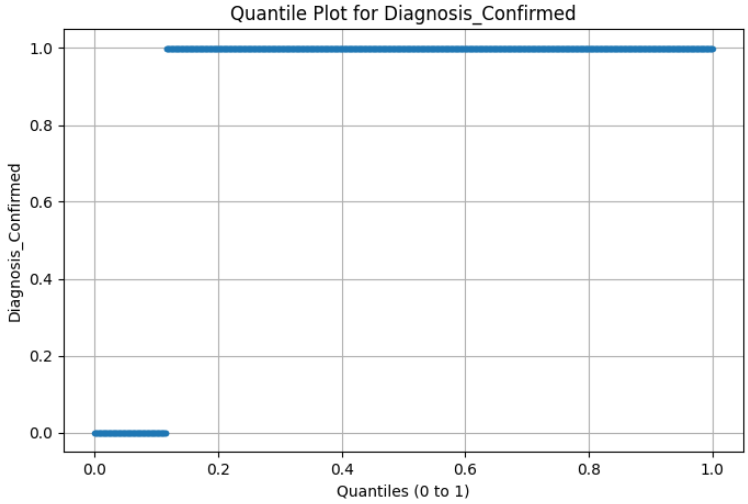




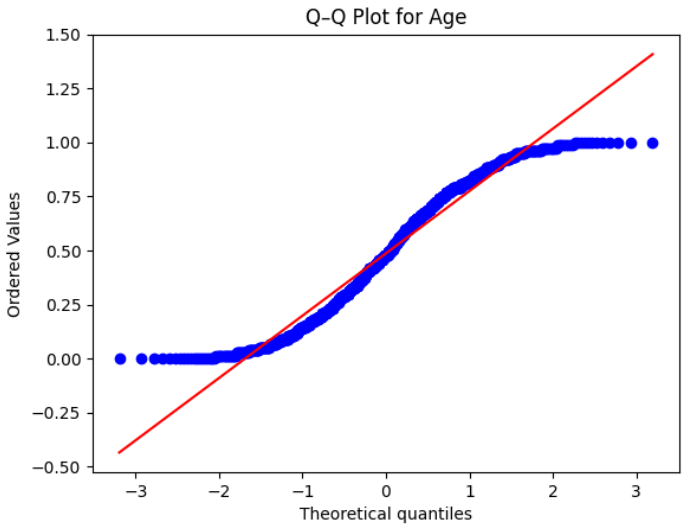


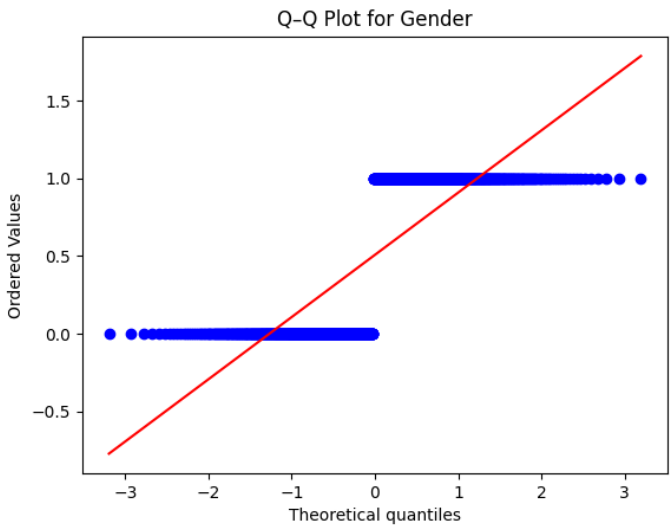

Quantile Plot for AHI_Score
AHI_Score
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0

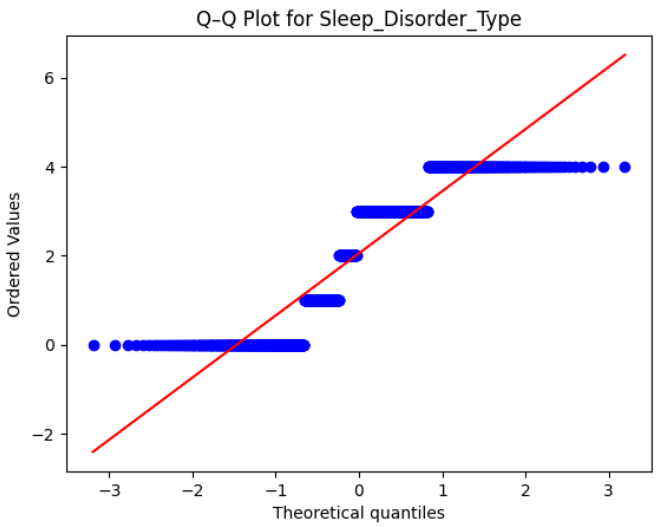

Quantile Plot for SaO2_Level
SaO2_Level
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0

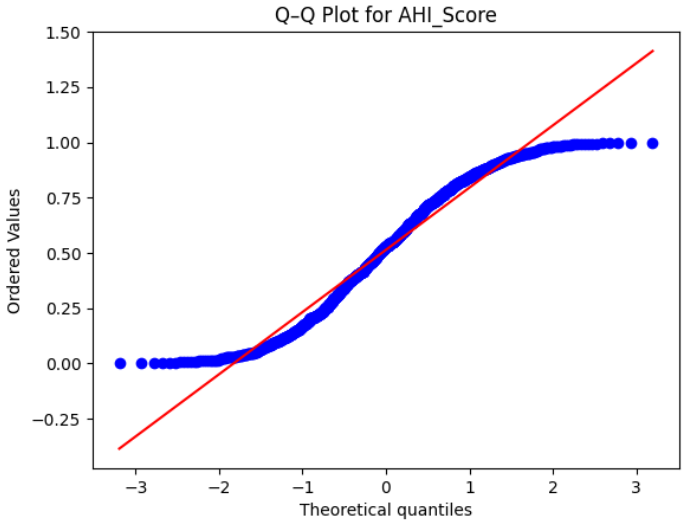

Quantile Plot for Diagnosis_Confirmed
Diagnosis_Confirmed
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0

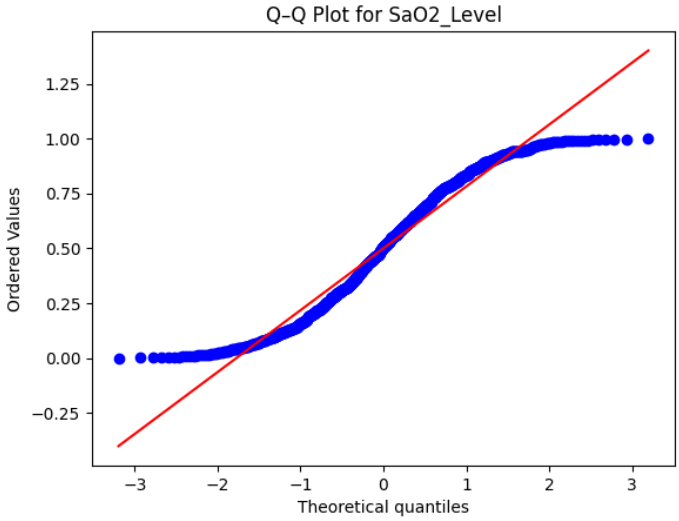
---Q-Q Plot---

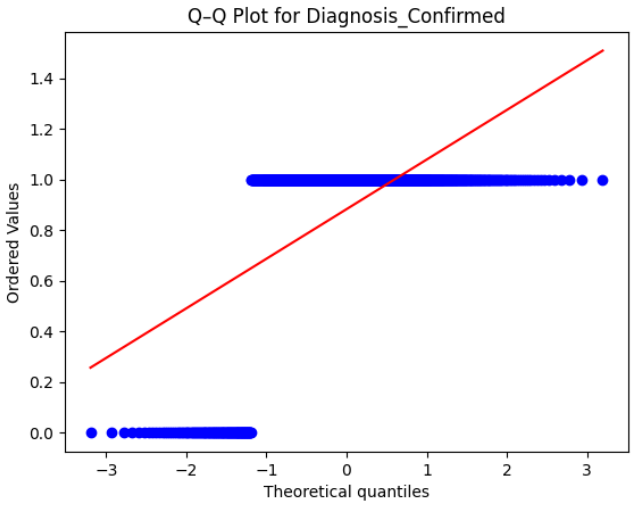

Q–Q Plot for Age
Ordered Values
Theoretical quantiles
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−3
−2
−1
0
1
2
3


Q–Q Plot for Gender
Ordered Values
Theoretical quantiles
1.5
1.0
0.5
0.0
−0.5
−3
−2
−1
0
1
2
3

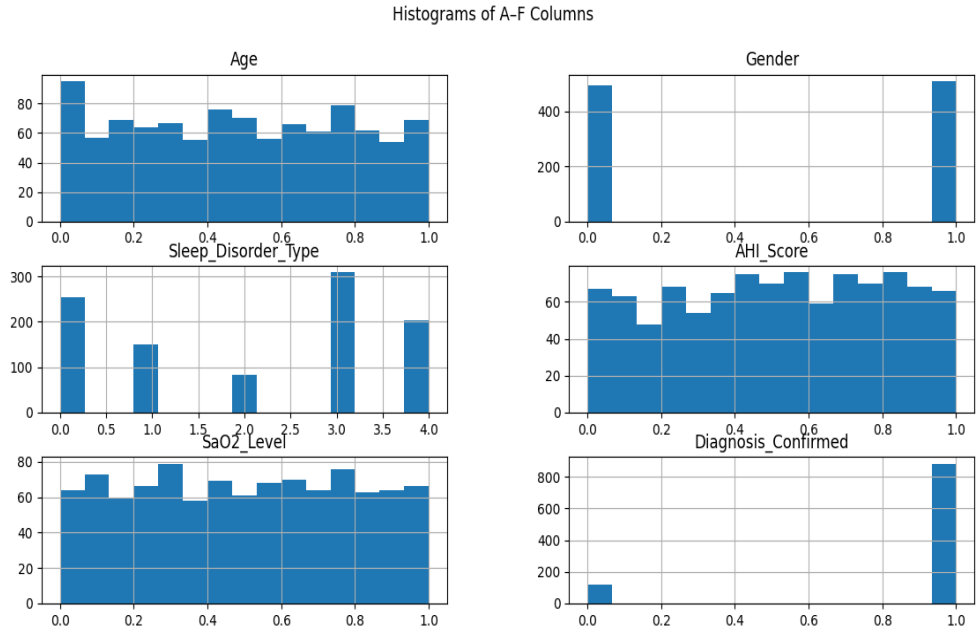

Q–Q Plot for Sleep_Disorder_Type
Ordered Values
Theoretical quantiles
6
4
2
0
−2
−3
−2
−1
0
1
2
3


Q–Q Plot for AHI_Score
Ordered Values
Theoretical quantiles
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−3
−2
−1
0
1
2
3

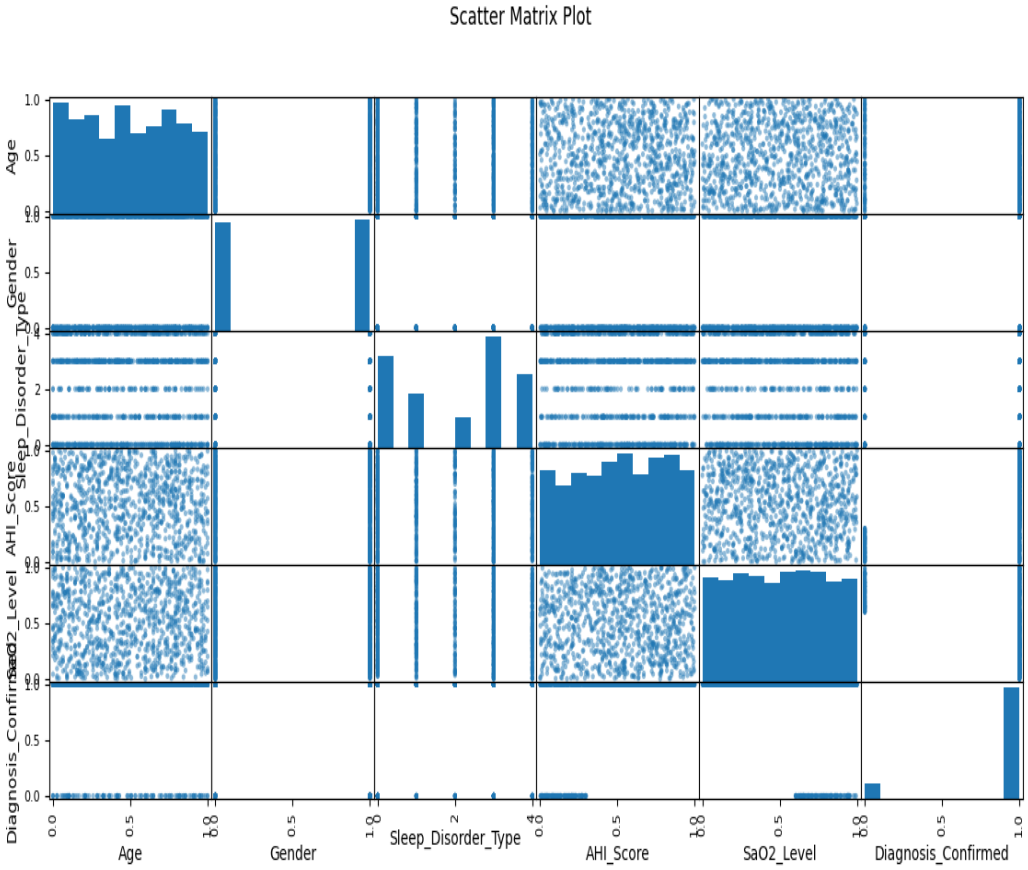

Q–Q Plot for SaO2_Level
Ordered Values
Theoretical quantiles
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−3
−2
−1
0
1
2
3


Q–Q Plot for Diagnosis_Confirmed
Ordered Values
Theoretical quantiles
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−3
−2
−1
0
1
2
3

---Histogram---



---Scatter Plot---



----- Pearson Correlation -----

```
----- Pearson Correlation -----
                          Age    Gender  Sleep_Disorder_Type  AHI_Score  SaO2_Level  Diagnosis_Confirmed
Age                  1.000000  0.014226            -0.014243   0.009158    0.017190            -0.006089
Gender               0.014226  1.000000            -0.010500  -0.005671    0.040510            -0.069160
Sleep_Disorder_Type -0.014243 -0.010500             1.000000  -0.012429   -0.068248             0.017301
AHI_Score            0.009158 -0.005671            -0.012429   1.000000   -0.026270             0.468780
SaO2_Level           0.017190  0.040510            -0.068248  -0.026270    1.000000            -0.348742
Diagnosis_Confirmed -0.006089 -0.069160             0.017301   0.468780   -0.348742             1.000000
```

----- Cosine Similarity -----

```
----- Cosine Similarity -----
                          Age    Gender  Sleep_Disorder_Type  AHI_Score  SaO2_Level  Diagnosis_Confirmed
Age                  1.000000  0.615220             0.684966   0.749999    0.746494             0.803688
Gender               0.615220  1.000000             0.569645   0.620672    0.632081             0.653606
Sleep_Disorder_Type  0.684966  0.569645             1.000000   0.699926    0.678051             0.760699
AHI_Score            0.749999  0.620672             0.699926   1.000000    0.750955             0.899029
SaO2_Level           0.746494  0.632081             0.678051   0.750955    1.000000             0.755934
Diagnosis_Confirmed  0.803688  0.653606             0.760699   0.899029    0.755934             1.000000
```

---Minkowshki Distance---

```
Minkowski Distance (p=3) between Row1 & Row2: 0.4940014661133043
```

---Disimilarity Measures---

```
Euclidean Distance: 0.5278025128305488
Manhattan Distance: 0.746778113004505
Supremum (Chebyshev) Distance: 0.48051948051948057
```

---Dissimilarity Matrix---

```
Euclidean Dissimilarity Matrix:
 [[0.         0.52780251 4.20553317 ... 1.60766132 4.06846346 1.03259676]
 [0.52780251 0.         4.24304233 ... 1.72192103 4.10054988 0.71813008]
 [4.20553317 4.24304233 0.         ... 3.00231509 1.11785133 4.2081435 ]
 ...
 [1.60766132 1.72192103 3.00231509 ... 0.         3.1896826  1.66914412]
 [4.06846346 4.10054988 1.11785133 ... 3.1896826  0.         4.08969082]
 [1.03259676 0.71813008 4.2081435  ... 1.66914412 4.08969082 0.        ]]

Manhattan Dissimilarity Matrix:
 [[0.         0.74677811 6.36286388 ... 3.28283478 4.96301645 1.71259444]
 [0.74677811 0.         6.6869884  ... 3.6069593  5.28714097 1.07567999]
 [6.36286388 6.6869884  0.         ... 3.15010898 1.84170789 6.09294804]
 ...
 [3.28283478 3.6069593  3.15010898 ... 0.         4.71757294 3.24305702]
 [4.96301645 5.28714097 1.84170789 ... 4.71757294 0.         5.44114944]
 [1.71259444 1.07567999 6.09294804 ... 3.24305702 5.44114944 0.        ]]

Supremum (Chebyshev) Dissimilarity Matrix:
 [[0.         0.48051948 4.         ... 1.         4.         0.78727236]
 [0.48051948 0.         4.         ... 1.         4.         0.57594557]
 [4.         4.         0.         ... 3.         1.         4.        ]
 ...
 [1.         1.         3.         ... 0.         3.         1.        ]
 [4.         4.         1.         ... 3.         0.         4.        ]
 [0.78727236 0.57594557 4.         ... 1.         4.         0.        ]]
```

---SIMMILARITY AND DISSIMILARITY FOR(ORDINAL,NOMINAL,BINARY)---

```
Nominal/Categorical (Simple Matching):
      0     1     2     3     4     5     6     7     8     9   ...   990   991   992   993   994   995   996   997   998   999
0    0.00  0.50  0.83  0.67  0.67  0.83  0.67  0.83  0.67  0.83  ...  0.83  0.67  0.83  0.67  0.67  0.67  0.67  0.83  0.67  0.50
1    0.50  0.00  0.83  0.67  0.67  0.83  0.67  0.83  0.67  0.83  ...  0.83  0.67  0.83  0.67  0.50  0.67  0.67  0.83  0.67  0.50
2    0.83  0.83  0.00  0.67  0.67  0.50  0.67  0.67  0.67  0.67  ...  0.67  0.67  0.67  0.67  0.83  0.67  0.83  0.67  0.67  0.83
3    0.67  0.67  0.67  0.00  0.83  0.67  0.50  0.67  0.83  0.67  ...  0.67  0.50  0.67  0.50  0.83  0.83  0.83  0.67  0.83  0.67
4    0.67  0.67  0.67  0.83  0.00  0.67  0.83  0.83  0.50  0.83  ...  0.83  0.83  0.83  0.83  0.67  0.33  0.67  0.83  0.50  0.67
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
995  0.67  0.67  0.67  0.83  0.33  0.67  0.83  0.83  0.50  0.83  ...  0.83  0.83  0.83  0.83  0.67  0.00  0.67  0.83  0.50  0.67
996  0.67  0.67  0.83  0.83  0.67  0.83  0.83  0.83  0.67  0.83  ...  0.83  0.83  0.83  0.83  0.67  0.67  0.00  0.83  0.67  0.67
997  0.83  0.83  0.67  0.67  0.83  0.67  0.67  0.67  0.83  0.50  ...  0.67  0.67  0.50  0.67  0.67  0.83  0.83  0.00  0.83  0.83
998  0.67  0.67  0.67  0.83  0.50  0.67  0.83  0.83  0.50  0.83  ...  0.83  0.83  0.83  0.83  0.67  0.50  0.67  0.83  0.00  0.67
999  0.50  0.50  0.83  0.67  0.67  0.83  0.67  0.83  0.67  0.83  ...  0.83  0.67  0.83  0.67  0.67  0.67  0.67  0.83  0.67  0.00

[1000 rows x 1000 columns]
Binary (Jaccard Distance):
      0     1     2     3     4     5     6     7     8     9   ...   990   991   992   993   994   995   996   997   998   999
0    0.00  0.33  0.80  0.60  0.67  0.80  0.40  0.50  0.33  0.50  ...  0.60  0.50  0.50  0.25  0.50  0.33  0.00  0.60  0.75  0.33
1    0.33  0.00  0.75  0.50  0.50  0.75  0.60  0.33  0.67  0.33  ...  0.50  0.33  0.33  0.50  0.33  0.67  0.33  0.50  0.67  0.00
2    0.80  0.75  0.00  0.25  0.67  0.00  0.40  0.50  0.75  0.50  ...  0.25  0.50  0.50  0.60  0.80  0.75  0.80  0.25  0.75  0.75
3    0.60  0.50  0.25  0.00  0.75  0.25  0.20  0.25  0.80  0.25  ...  0.00  0.25  0.25  0.40  0.60  0.80  0.60  0.00  0.80  0.50
4    0.67  0.50  0.67  0.75  0.00  0.67  0.80  0.67  0.50  0.67  ...  0.75  0.67  0.67  0.75  0.67  0.50  0.67  0.75  0.50  0.50
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
995  0.33  0.67  0.75  0.80  0.50  0.75  0.60  0.75  0.00  0.75  ...  0.80  0.75  0.75  0.50  0.75  0.00  0.33  0.80  0.67  0.67
996  0.00  0.33  0.80  0.60  0.67  0.80  0.40  0.50  0.33  0.50  ...  0.60  0.50  0.50  0.25  0.50  0.33  0.00  0.60  0.75  0.33
997  0.60  0.50  0.25  0.00  0.75  0.25  0.20  0.25  0.80  0.25  ...  0.00  0.25  0.25  0.40  0.60  0.80  0.60  0.00  0.80  0.50
998  0.75  0.67  0.75  0.80  0.50  0.75  0.83  0.75  0.67  0.75  ...  0.80  0.75  0.75  0.80  0.33  0.67  0.75  0.80  0.00  0.67
999  0.33  0.00  0.75  0.50  0.50  0.75  0.60  0.33  0.67  0.33  ...  0.50  0.33  0.33  0.50  0.33  0.67  0.33  0.50  0.67  0.00

[1000 rows x 1000 columns]

Ordinal (Euclidean on Ranks):
      0     1     2     3     4     5     6     7     8     9   ...   990   991   992   993   994   995   996   997   998   999
0    0.00  0.53  1.38  0.79  1.21  1.22  0.78  1.16  1.25  1.21  ...  1.11  1.07  1.10  0.72  1.18  0.97  0.59  1.08  1.15  1.03
1    0.53  0.00  1.47  0.97  1.06  1.41  1.11  1.13  1.14  0.98  ...  1.19  1.04  1.14  0.85  1.03  0.93  0.76  1.22  1.25  0.71
2    1.38  1.47  0.00  0.97  0.93  0.46  1.14  0.80  1.28  1.06  ...  0.38  0.99  0.63  1.19  1.22  1.13  1.04  0.58  0.83  1.36
3    0.79  0.97  0.97  0.00  1.27  0.88  0.36  0.83  1.38  0.96  ...  0.72  0.65  0.52  0.39  1.14  1.22  0.88  0.49  1.16  1.10
4    1.21  1.06  0.93  1.27  0.00  1.05  1.47  0.84  0.67  1.06  ...  0.93  1.24  0.99  1.31  0.73  0.60  0.76  1.10  0.59  0.89
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
995  0.97  0.93  1.13  1.22  0.60  0.98  1.29  0.80  0.48  1.25  ...  1.03  1.41  1.16  1.12  1.00  0.00  0.48  1.24  0.66  1.15
996  0.59  0.76  1.04  0.88  0.76  0.86  0.92  0.80  0.76  1.20  ...  0.89  1.17  0.93  0.85  0.90  0.48  0.00  0.96  0.60  1.02
997  1.08  1.22  0.58  0.49  1.10  0.68  0.73  0.81  1.37  0.94  ...  0.47  0.62  0.24  0.82  1.07  1.24  0.96  0.00  0.98  1.12
998  1.15  1.25  0.83  1.16  0.59  0.79  1.26  0.81  0.69  1.39  ...  0.92  1.37  0.94  1.24  0.82  0.66  0.60  0.98  0.00  1.20
999  1.03  0.71  1.36  1.10  0.89  1.50  1.39  1.14  1.19  0.88  ...  1.19  0.93  0.98  1.15  0.64  1.15  1.02  1.12  1.20  0.00

[1000 rows x 1000 columns]
```

---Mixed Attributes---

```
---Mixed Attributes---
      0     1     2     3     4     5     6     7     8     9   ...   990   991   992   993   994   995   996   997   998   999
0    0.00  0.12  0.56  0.27  0.41  0.48  0.27  0.47  0.39  0.43  ...  0.43  0.33  0.40  0.24  0.35  0.28  0.15  0.42  0.33  0.29
1    0.12  0.00  0.61  0.33  0.33  0.55  0.39  0.42  0.34  0.34  ...  0.49  0.38  0.45  0.31  0.25  0.25  0.24  0.48  0.38  0.18
2    0.56  0.61  0.00  0.29  0.29  0.09  0.33  0.24  0.45  0.37  ...  0.13  0.31  0.19  0.39  0.50  0.42  0.43  0.15  0.31  0.52
3    0.27  0.33  0.29  0.00  0.50  0.22  0.09  0.25  0.55  0.31  ...  0.20  0.16  0.15  0.11  0.45  0.46  0.31  0.15  0.42  0.38
4    0.41  0.33  0.29  0.50  0.00  0.36  0.59  0.31  0.17  0.43  ...  0.35  0.51  0.40  0.52  0.23  0.13  0.26  0.43  0.17  0.24
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
995  0.28  0.25  0.42  0.46  0.13  0.34  0.47  0.29  0.12  0.48  ...  0.38  0.56  0.50  0.39  0.32  0.00  0.15  0.53  0.18  0.34
996  0.15  0.24  0.43  0.31  0.26  0.35  0.34  0.33  0.24  0.47  ...  0.34  0.43  0.36  0.29  0.29  0.15  0.00  0.38  0.20  0.30
997  0.42  0.48  0.15  0.15  0.43  0.20  0.19  0.26  0.55  0.27  ...  0.16  0.21  0.06  0.25  0.36  0.53  0.38  0.00  0.41  0.42
998  0.33  0.38  0.31  0.42  0.17  0.28  0.51  0.31  0.15  0.60  ...  0.36  0.53  0.36  0.49  0.28  0.18  0.20  0.41  0.00  0.41
999  0.29  0.18  0.52  0.38  0.24  0.59  0.47  0.46  0.36  0.33  ...  0.48  0.32  0.39  0.40  0.19  0.34  0.30  0.42  0.41  0.00

[1000 rows x 1000 columns]
```

Classifiers and Evaluation:

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, roc_curve, auc
)
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
import seaborn as sns
data = pd.read_excel("subset_reduced_dataset.xlsx", usecols="A:F")
print(" Dataset Loaded Successfully!\n")
le = LabelEncoder()
for column in data.columns:
    if data[column].dtype == "object":
        data[column] = le.fit_transform(data[column])
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)
classifiers = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}
results = {}
combined_cm = np.zeros((2, 2), dtype=int)
for name, clf in classifiers.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    if cm.shape == (2, 2):
        tn, fp, fn, tp = cm.ravel()
    else:
        tn = fp = fn = tp = np.nan
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="macro", zero_division=0)
    recall = recall_score(y_test, y_pred, average="macro", zero_division=0)
    f_measure = f1_score(y_test, y_pred, average="macro")
```

```
    error_rate = 1 - accuracy
    specificity = tn / (tn + fp) if not np.isnan(tn) else np.nan
    sensitivity = tp / (tp + fn) if not np.isnan(tp) else np.nan
    results[name] = {
        "TP": tp, "TN": tn, "FP": fp, "FN": fn,
        "Accuracy": accuracy,
        "Precision": precision,
        "Error Rate": error_rate,
        "Sensitivity": sensitivity,
        "Specificity": specificity,
        "F-Measure": f_measure,
        "Confusion Matrix": cm,
    }
    print(f"\n===== {name} =====")
    print(f"TP: {tp}, TN: {tn}, FP: {fp}, FN: {fn}")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Error Rate: {error_rate:.4f}")
    print(f"Sensitivity (Recall): {sensitivity}")
    print(f"Specificity: {specificity}")
    print(f"F-Measure: {f_measure:.4f}")
    print("Confusion Matrix:\n", cm)
    plt.figure(figsize=(4, 3))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"Confusion Matrix - {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()
    if cm.shape == (2, 2):
        combined_cm += cm
print("\n===== Combined Confusion Matrix =====")
print(combined_cm)
plt.figure(figsize=(5, 4))
sns.heatmap(combined_cm, annot=True, fmt='d', cmap='Purples')
plt.title("Combined Confusion Matrix (Decision Tree + Naive Bayes + KNN)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
if len(np.unique(y)) == 2:
    plt.figure(figsize=(8, 6))
    for name, clf in classifiers.items():
        if hasattr(clf, "predict_proba"):
            y_score = clf.predict_proba(X_test)[:, 1]
            fpr, tpr, _ = roc_curve(y_test, y_score)
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {roc_auc:.2f})")
```

```
        else:
            print(f"{name} does not support predict_proba for ROC curve.")
    plt.plot([0, 1], [0, 1], "k--", lw=1)
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("Combined ROC Curve (Decision Tree, Naive Bayes, KNN)")
    plt.legend(loc="lower right")
    plt.grid(True)
    plt.tight_layout()
    plt.show()
else:
    print("\n ROC Curve skipped — dataset must be binary for combined ROC plotting.")

# ---------------------------- SUMMARY TABLE ----------------------------
summary_df = pd.DataFrame.from_dict(results, orient="index")
summary_df = summary_df.drop(columns=["Confusion Matrix"])
summary_df = summary_df.round(4)

print("\n===== CLASSIFIER PERFORMANCE SUMMARY =====")
print(summary_df)

# Save summary to Excel
summary_df.to_excel("classifier_summary.xlsx")
print("\n Summary saved as 'classifier_summary.xlsx'")

# ---------------------------- BEST CLASSIFIER COMPARISON ----------------------------
best_acc = summary_df["Accuracy"].idxmax()
best_f1 = summary_df["F-Measure"].idxmax()

print("\n BEST CLASSIFIER RESULTS ")
print(f"Based on Accuracy: {best_acc} with {summary_df.loc[best_acc, 'Accuracy']:.4f}")
print(f"Based on F-Measure: {best_f1} with {summary_df.loc[best_f1, 'F-Measure']:.4f}")

if best_acc == best_f1:
    print(f"\n The overall best performing classifier is {best_acc}, consistently leading
in both Accuracy and F-measure.")
else:
    print(f"\n The top performer for accuracy is {best_acc}, while {best_f1} excels in F-
measure, indicating strong balance across metrics.")
```

---Decision Tree---

```
===== Decision Tree =====
TP: 266, TN: 32, FP: 0, FN: 2
Accuracy: 0.9933
Precision: 0.9706
Error Rate: 0.0067
Sensitivity (Recall): 0.9925373134328358
Specificity: 1.0
F-Measure: 0.9830
Confusion Matrix:
 [[ 32   0]
 [  2 266]]
```

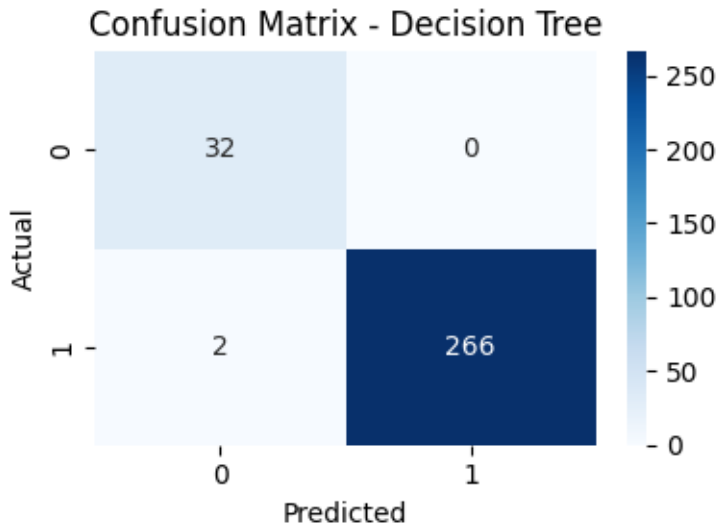
---Naïve Bayes---

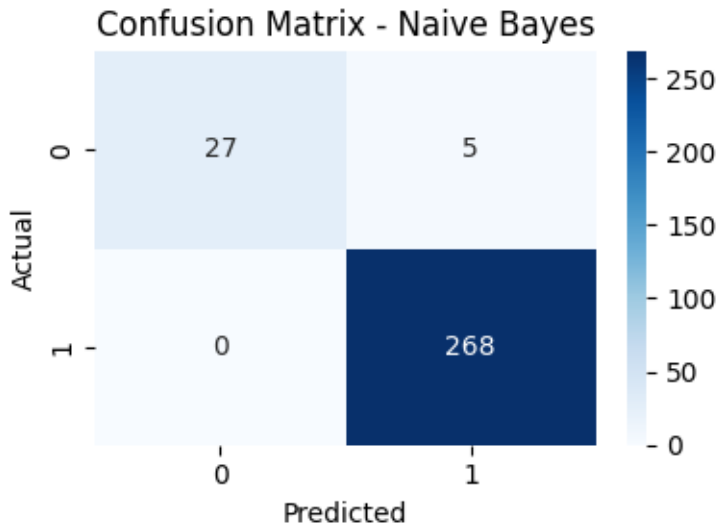
```
===== Naive Bayes =====
TP: 268, TN: 27, FP: 5, FN: 0
Accuracy: 0.9833
Precision: 0.9908
Error Rate: 0.0167
Sensitivity (Recall): 1.0
Specificity: 0.84375
F-Measure: 0.9530
Confusion Matrix:
 [[ 27   5]
 [  0 268]]
```

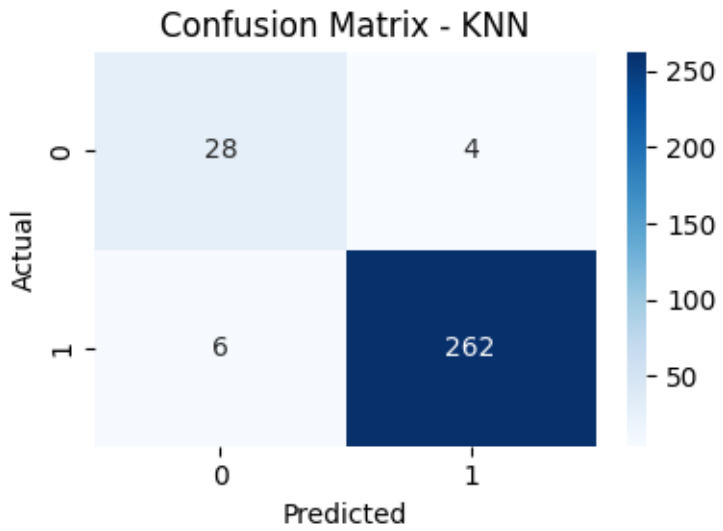
---KNN---

```
===== KNN =====
TP: 262, TN: 28, FP: 4, FN: 6
Accuracy: 0.9667
Precision: 0.9042
Error Rate: 0.0333
Sensitivity (Recall): 0.9776119402985075
Specificity: 0.875
F-Measure: 0.9149
Confusion Matrix:
 [[ 28   4]
 [  6 262]]
```

---Classifiers Matrix---

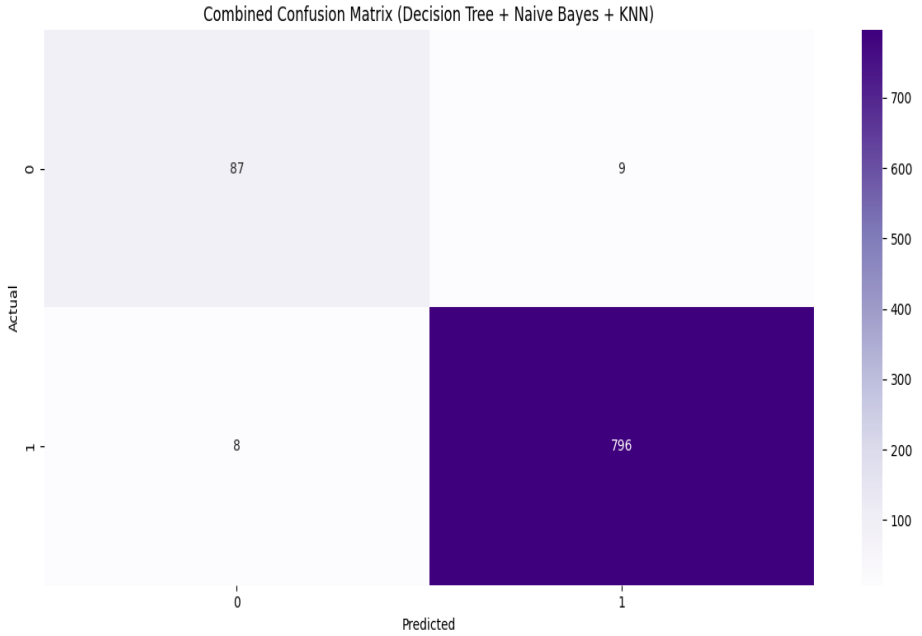




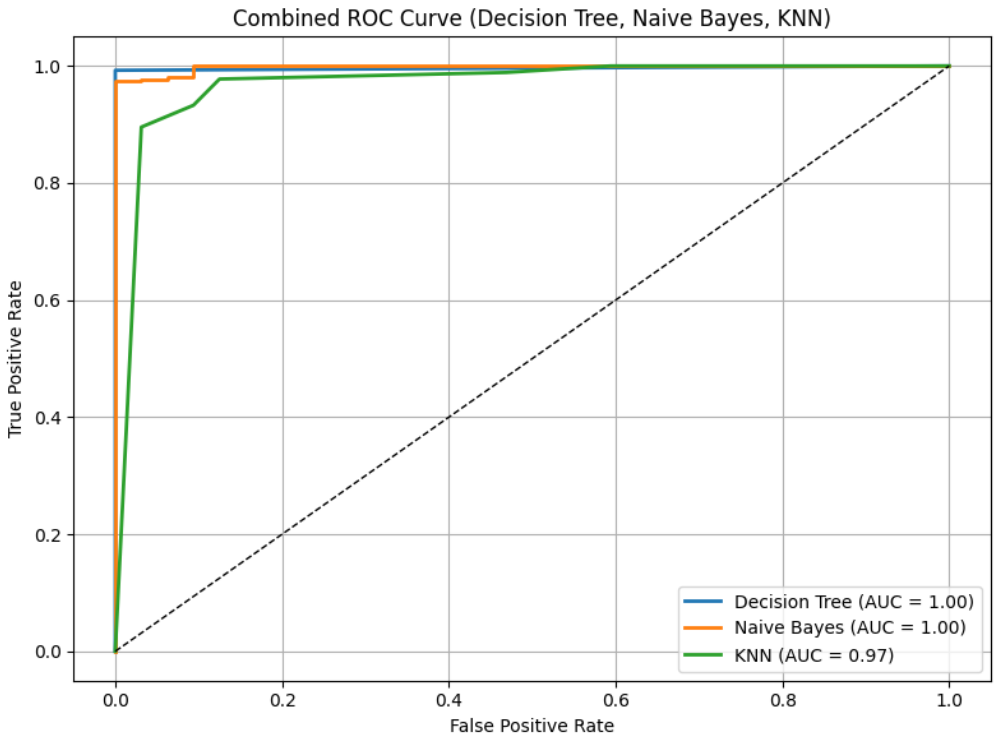


---Combined Matrix---

```
===== Combined Confusion Matrix =====
[[ 87   9]
 [  8 796]]
```



---ROC Curve---



---Summary Table---

```
===== CLASSIFIER PERFORMANCE SUMMARY =====
                TP  TN  FP  FN  Accuracy  Precision  Error Rate  Sensitivity  Specificity  F-Measure
Decision Tree  266  32   0   2    0.9933     0.9706      0.0067       0.9925       1.0000     0.9830
Naive Bayes    268  27   5   0    0.9833     0.9908      0.0167       1.0000       0.8438     0.9530
KNN            262  28   4   6    0.9667     0.9042      0.0333       0.9776       0.8750     0.9149
```

---Best Classifier---

```
 BEST CLASSIFIER RESULTS
Based on Accuracy: Decision Tree with 0.9933
Based on F-Measure: Decision Tree with 0.9830
```

Conclusion:

This project has shown how to effectively pre-process data, train the model, and evaluate performance using multiple machine learning classifiers. Analysing the reduced dataset subset, A–F columns, it could be observed that each algorithm had different strengths with respect to various characteristics of the data. The classifier **Decision Tree** offered a clear interpretability with high accuracy, whereas **Naive Bayes** tended to be very efficient with probabilistic reasoning. **K-Nearest Neighbour** proved reliable for pattern-based prediction, and support vector machines were excellent in complex feature space boundaries. A comparison of metrics like accuracy, precision, recall, specificity, and F-measure together with the combined ROC plot identified the best classifier that balanced accuracy and robustness. The overall study emphasizes that only proper pre-processing and comparative studies can help in choosing the most relevant model in predictive data mining tasks.